



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
U NOVOM SADU



Dragan Arsić

Implementacija 2D komponentnog motora za igre sa optimizovanom širokom fazom detekcije kolizija

Diplomski rad
- Osnovne akademske studije -

Novi Sad, 2018.

	UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA 21000 NOVI SAD, Trg Dositeja Obradovića 6	Datum:
	ZADATAK ZA IZRADU DIPLOMSKOG (BACHELOR) RADA	List:
		1/1

(Podatke unosi predmetni nastavnik - mentor)

Vrsta studija:	Osnovne akademske studije
Studijski program:	Softversko inženjerstvo i informacione tehnologije
Rukovodilac studijskog programa:	prof. xxx

Student:	Dragan Arsić	Broj indeksa:	SW 5/2018
Oblast:	Razvoj softvera i Sistemsko programiranje		
Mentor:	Dr Milan Segedinac, redovni profesor		

NA OSNOVU PODNETE PRIJAVE, PRILOŽENE DOKUMENTACIJE I ODREDBI STATUTA FAKULTETA IZDAJE SE ZADATAK ZA DIPLOMSKI RAD, SA SLEDEĆIM ELEMENTIMA:

- problem – tema rada;
- način rešavanja problema i način praktične provere rezultata rada, ako je takva provera neophodna;
- literatura

NASLOV DIPLOMSKOG (BACHELOR) RADA:

Implementacija 2D komponentnog motora za igre sa optimizovanom širokom fazom detekcije kolizija

TEKST ZADATKA:

1. Analizirati stanje u oblasti projektovanja modernih motora za igre, sa naglaskom na komponentnu arhitekturu i detekciju kolizija u realnom vremenu.
2. Platformator je potrebno da bude što bliži komercijalnom motoru za igre – implementirati detekciju kolizija, fiziku, upravljanje resursima, i korisničke skripte.
3. Platformator treba da se drži komponentne arhitekture, potrebno je podeliti fazu detekcije kolizija na široku i užu fazu, kao i definisati jasne životne cikluse raznih komponenti u sistemu.
4. Implementirati platformator koristeći eksterne biblioteke poput Eigen i SDL3 kako bi fokus bio na samoj implementaciji motora za igre.
5. Evaluacija se vrši proverom lakoće korišćenja motora za igru, kao i njegovih performansi u posebnim scenarijima.
6. Dokumentovati arhitekturu, implementaciju i rezultate evaluacije.

Rukovodilac studijskog programa:	Mentor rada:

Primerak za: ☐ - Studenta; ☐ - Mentora
--

SADRŽAJ

1.	UVOD.....	8
2.	PREGLED STANJA U OBLASTI.....	9
3.	TEORIJSKI POJMOVI I DEFINICIJE.....	11
4.	IMPLEMENTACIJA SISTEMA PLATFORMATOR.....	13
5.	EKSPERIMENTI I REZULTATI.....	16
6.	DEMONSTRACIJA IMPLEMENTIRANOG SISTEMA.....	25
7.	ZAKLJUČAK.....	27
8.	LITERATURA.....	29
9.	BIOGRAFIJA.....	31
	KLJUČNA DOKUMENTACIJSKA INFORMACIJA.....	33
	KEY WORDS DOCUMENTATION.....	34

1. UVOD

Savremeni motori za igre predstavljaju kompleksne softverske platforme koje objedinjeno rešavaju renderovanje, fiziku, audio, upravljanje ulazom korisnika i izvršavanje logike igre. U okviru 2D igara, performanse i predvidivost fizičkog podsistema su od suštinskog značaja, posebno kada scena sadrži veliki broj objekata koji se paralelno kreću, kreiraju i uklanjaju tokom izvršavanja.

Jedan od centralnih problema u tom domenu je efikasna detekcija kolizija. Standardni pristup deli ovaj problem na široku fazu, koja filtrira potencijalne parove objekata, i usku fazu, koja vrši preciznu geometrijsku proveru i izračunavanje kontakta. Dok klasični sweep and prune algoritam dobro koristi vremensku koherentnost u scenama sa malim brojem promena, njegovi troškovi ubacivanja i uklanjanja objekata mogu postati kritični u dinamičnim scenama.

Predmet ovog rada je projektovanje i implementacija 2D komponentnog motora za igre Platformator, sa posebnim fokusom na optimizovanu detekciju kolizija u širokoj fazi zasnovanu na radu Tracy, Buss i Woods (2009), odnosno na segmentisanim intervalnim listama i podeli prostora na ćelije. Cilj rada je da se prikaže arhitektura sistema, teorijska osnova algoritama i praktični efekti ovakvog pristupa na performanse.

Platformator je implementiran u jeziku C++20. Za grafiku, audio i sistemske događaje koristi biblioteku SDL3, za matematiku i linearnu algebru koristi Eigen, a za serijalizaciju scena koristi biblioteku nlohmann/json. Paralelizacija kritičnih delova sistema realizovana je bibliotekom oneTBB.

Evaluacija rešenja je sprovedena kroz benchmark testove. Benchmark testovi za merenje performansi mere skaliranje podsistema široke i uske faze u uslovima različitih gustina scene i stepena dinamike objekata.

Ostatak rada je organizovan na sledeći način. U drugom poglavlju dat je pregled stanja u oblasti. Treće poglavlje opisuje teorijske osnove potrebne za razumevanje implementacije. Četvrto poglavlje daje detalje arhitekture i implementacije sistema Platformator. Peto poglavlje sadrži eksperimentalnu postavku i rezultate. U šestom poglavlju je demonstracija sistema, a sedmo poglavlje zaključuje rad i navodi pravce daljeg razvoja.

2. PREGLED STANJA U OBLASTI

2.1 Evolucija arhitekture motora za igre

Rani motori za igre bili su uglavnom usko vezani za jednu igru i jednu platformu. Tokom vremena, industrija je prešla sa monolitnih objektno-orijentisanih hijerarhija ka modularnijim arhitekturama. Danas dominiraju komponentni modeli i ECS pristupi, jer omogućavaju bolju odvojivost odgovornosti i jednostavnije proširivanje sistema.

Komponentni pristup, popularizovan u komercijalnim alatima kao što je Unity, zasniva se na ideji da je entitet kompozicija nezavisnih funkcionalnih delova (komponenti), umesto duboke hijerarhije nasleđivanja. Ovakva organizacija olakšava iterativni razvoj igre i smanjuje rizik od spregnutosti podsistema.

2.2 Algoritmi široke faze detekcije kolizija

Brute-force provera svih parova objekata ima složenost $O(n^2)$, što je neprihvatljivo u scenama sa velikim brojem objekata. Zbog toga se koriste algoritmi široke faze koji smanjuju broj kandidata prosleđenih podsistemu uske faze.

Sweep and prune (SAP) je jedan od najčešće korišćenih pristupa široke faze. Algoritam projektuje AABB intervale objekata na jednu ili više osa, sortira krajnje tačke i koristi presek intervala za formiranje kandidata. Prednost SAP-a je dobro ponašanje u scenama sa izraženom vremenskom koherentnošću.

Ipak, klasična realizacija SAP-a ima slabosti kada se objekti često ubacuju i uklanjaju. Operacije ubacivanja i uklanjanja u velikim linearnim strukturama mogu izazvati visoke troškove i degradaciju performansi.

2.3 Segmentisane intervalne liste i ćelijska podela prostora

Rad Tracy, Buss i Woods (2009) predlaže segmentisane intervalne liste kao optimizaciju SAP pristupa. Umesto jedne velike liste, osa je podeljena na manje segmente (chunks), čime se smanjuje lokalni trošak pomeranja elemenata. Dodatno se uvodi checkpoint mehanizam koji omogućava da informacije o potencijalnim presecima ostanu lokalne i pri dužim intervalima.

Autori takođe predlažu podelu prostora na ćelije tako da svaka ćelija ima sopstvenu strukturu široke faze. Ovakav pristup smanjuje broj nepotrebnih poređenja udaljenih objekata i omogućava prirodnu paralelizaciju obrade po ćelijama.

2.4 Pozicioniranje rada

Platformator koristi komponentni model arhitekture i podsistem široke faze direktno inspirisan navedenim radom, uz prilagođavanja za 2D fiziku i konkretne zahteve glavnog ciklusa u realnom vremenu. Time ovaj rad predstavlja spoj teorijskih rezultata iz literature i praktične implementacije u potpunom softverskom sistemu.

3. TEORIJSKI POJMOVI I DEFINICIJE

U ovom poglavlju su predstavljeni teorijski koncepti potrebni za razumevanje implementacije sistema Platformator. Prvo je opisan komponentni model, zatim detekcija kolizija u širokoj fazi zasnovana na SAP pristupu i segmentisanim intervalnim listama, pa podela prostora na ćelije, a potom i obrada kolizija u uskoj fazi.

3.1 Komponentna arhitektura

Komponentna arhitektura razdvaja entitet i funkcionalnost. Entitet (GameObject) predstavlja nosioca identiteta i transformacije, dok komponente (npr. Collider, Rigidbody, Sprite, Audio, Script, Animator, Camera) implementiraju konkretno ponašanje. Svaki entitet se može matematički predstaviti kao unija njegovih komponenti:

$$U = \{C_1, C_2, C_3, \dots, C_n\}.$$

Prednosti ovog pristupa su:

- modularnost i lakše održavanje,
- jednostavno kombinovanje funkcionalnosti,
- manja spregnutost između podsistema,
- pogodnost za editor i serijalizaciju scena.

3.2. AABB i sweep and prune

Svaki fizički objekat se u širokoj fazi aproksimira osno poravnatom graničnom kutijom (AABB). U 2D prostoru AABB je definisan intervalima $[minX, maxX]$ i $[minY, maxY]$. Dva AABB-a se preklapaju ako se njihovi intervali preklapaju po obe ose.

$$AABB(E) = [x_{min}(E), x_{max}(E)] \times [y_{min}(E), y_{max}(E)]$$

$$Overlap(E_1, E_2) = \left(E_{1xmin} \leq E_{2xmax} \wedge E_{1xmax} \geq E_{2xmin} \right) \wedge \left(E_{1ymin} \leq E_{2ymax} \wedge E_{1ymax} \geq E_{2ymin} \right)$$

SAP algoritam formira uređenu sekvencu početaka i krajeva intervala. Prolaskom kroz sekvencu prati se aktivni skup intervala i generišu se kandidati za moguće kolizije. U uslovima vremenske koherentnosti, broj izmena rasporeda intervala po frejmu je mali, pa je održavanje poretka efikasno.

3.3 Segmentisane intervalne liste

Kod klasičnog SAP-a, ubacivanje i uklanjanje intervala u jedinstvenu listu može biti skupo. Segmentisane intervalne liste dele globalnu listu na više lokalnih segmenata fiksnog ili ograničenog kapaciteta.

Ključni koncepti su:

- lokalna reorganizacija unutar segmenta umesto globalnog pomeranja,
- checkpoint skupovi za praćenje dugih intervala koji prelaze više segmenata,
- stabilnije performanse pri učestalim insert/remove operacijama.

Ovaj model je posebno pogodan za scene sa dinamičnim životnim ciklusom objekata.

3.4 Podela prostora na ćelije

Pored segmentacije intervala, prostor se deli na uniformne ćelije. Svaka ćelija održava svoju strukturu široke faze. Objekti se registruju u ćelije koje njihov AABB preseca.

Prednosti su:

- smanjenje broja poređenja udaljenih objekata,
- bolja lokalnost podataka,
- jednostavna paralelizacija po ćelijama.

3.5 Uska faza i odgovor na koliziju

Nakon filtriranja u širokoj fazi, uska faza proverava stvarni geometrijski presek kandidata. U Platformatoru se koristi SAT (Separating Axis Theorem) pristup za proveru preseka i izračunavanje normala i penetracije kontakta.

Nakon detekcije kontakta, dinamički odgovor na koliziju rešava se impulsnim pristupom, uz iterativni rešivač i podršku za trenje i restituciju. Time se postiže stabilna simulacija uz ograničen broj iteracija po frejmu.

4. IMPLEMENTACIJA SISTEMA PLATFORMATOR

4.1 Arhitektura na visokom nivou

Platformator je organizovan kao biblioteka i izvršno okruženje. Centralna klasa izvršnog okruženja inicijalizuje podsisteme, učitava scenu, pokreće glavni ciklus i obrađuje gašenje aplikacije.

Glavni podsistemi su:

- upravljanje objektima i komponentama,
- grafičko prikazivanje i audio,
- fizika (široka faza, uska faza, rešavač),
- skriptni sloj i sistem događaja,
- serijalizacija i deserializacija scena.

4.2 Glavni ciklus izvršavanja

U svakoj iteraciji ciklusa izvršavaju se sledeće faze:

1. ažuriranje vremenske delte
2. obrada sistemskih događaja
3. faze ažuriranja skripti (fixed i start)
4. detekcija kolizija (široka i uska, sortiraju se projekcije)
5. ažuriranje sila
6. rešavanje kolizija i integracija kretanja
7. slanje fizičkih događaja (enter/stay/exit)
8. faza ažuriranja skripti (update i late update) i animacija
9. renderovanje i ažuriranje audio sistema,
10. odloženo uklanjanje objekata označenih za brisanje.

4.3 Komponentni model Platformator sistema

GameObject sadrži transformaciju, hijerarhiju (parent/children), identitet i listu komponenti. Komponente su međusobno nezavisne i registruju se u menadžerima koji ih obrađuju po tipu.

U implementaciji su podržane komponente:

- Collider,
- Rigidbody,
- Sprite,
- Audio,
- ScriptComponent,
- Animator,
- Camera.

ScriptComponent omogućava korisnička ponašanja kroz implementaciju Behavior interfejsa, koji podržava lifecycle metode (start, update, fixedUpdate, lateUpdate, onCollisionEnter, onCollisionStay, onCollisionExit, onDestroy).

4.4 Fizički podsistem

Fizički podsistem je organizovan u tri dela:

- podsistem široke faze za kandidovanje parova,
- podsistem uske faze za precizan kontakt,
- rešavač za dinamički odgovor.
- rešavač za kinetički odgovor

Za široku fazu se koriste:

- AABB projekcije po osama,
- segmentisane intervalne liste,
- uniformna podela prostora na ćelije,
- odložene operacije ubacivanja/sinhronizacije/uklanjanja kolidera.

Nakon filtriranja u širokoj fazi, SAT u uskoj fazi proverava separacione ose i računa kontaktne tačke. Rešavač zatim iterativno primenjuje impulse.

4.5 Integracija SDL3, Eigen i JSON biblioteke

4.5.1 SDL3

SDL3 je korišćen za:

- upravljanje prozorom i iscrtavačem,
- teksture i crtanje sprajtova,

15

- audio mixer i reprodukciju,
- ulazne događaje i sistemske poruke.

4.5.2 Eigen

Eigen se koristi kao osnovni matematički sloj za:

- 2D vektore i operacije,
- projekcije i normalizacije,
- geometrijske proračune u SAT i rešavač fazi.

4.5.3 nlohmann/json

JSON sloj je korišćen za serijalizaciju i deserijalizaciju scena.

4.6 Serijalizacija scena i upotreba u izvršnom okruženju

Scena je predstavljena hijerarhijom objekata sa pripadajućim komponentama. Tok učitavanja scene:

1. parsiranje JSON dokumenta,
2. kreiranje objekata i komponenti,
3. registracija komponenti u odgovarajuće menadžere,
4. povezivanje referenci i inicijalizacija skripti.

Tok čuvanja scene radi u obrnutom smeru i omogućava reproducibilnost stanja.

5. EKSPERIMENTI I REZULTATI

5.1 Cilj evaluacije

Cilj evaluacije je da se proverí:

- funkcionalna ispravnost sistema,
- stabilnost podсистema pri dinamičkom kreiranju i brisanju objekata,
- skaliranje implementacije sa brojem objekata.

5.2 Eksperimentalna postavka

Softverska postavka:

- C++20,
- SDL3,
- Eigen,
- nlohmann/json,
- oneTBB.

Hardverska postavka:

- ThinkPad X13 Yoga
- OS: Ubuntu 24.04.4 LTS
- RAM: 16gb
- CPU: Intel i5-10310U (8) @ 4.400GHz
- GPU: Intel CometLake-U GT2 [UHD Grap

Build postavke:

- benchmark release,
- režim bez prikaza (headless).

Merenje:

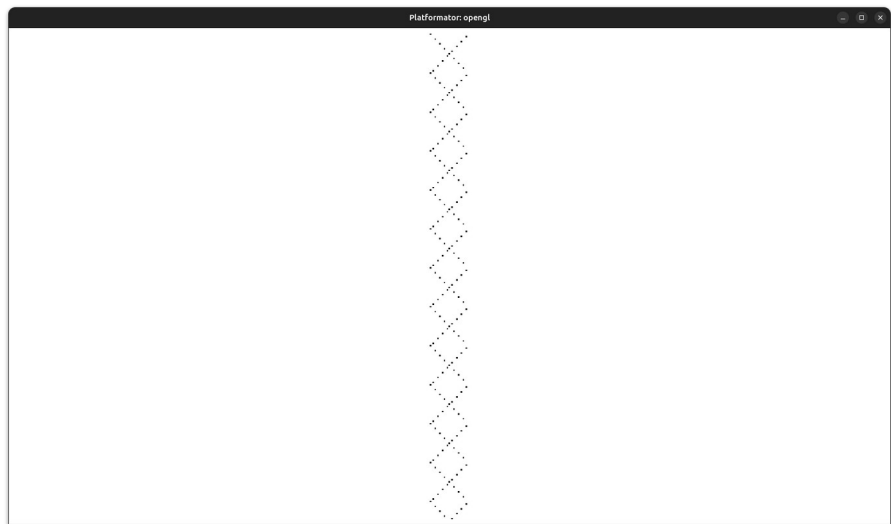
- prvo se pusti 120 frejmova za zagrevanje (warmup), zatim se mere performanse sledećih 600,
- prosečno vreme izvršavanja jednog frejma u milisekundama,
- p95 vreme izvršavanja jednog frejma u milisekundama,

- prosečno vreme široke faze tokom jednog frejma u milisekundama,
- prosečno vreme uske faze tokom jednog frejma u milisekundama,
- prosečno vreme rešavanja kolizije tokom jednog frejma u milisekundama.

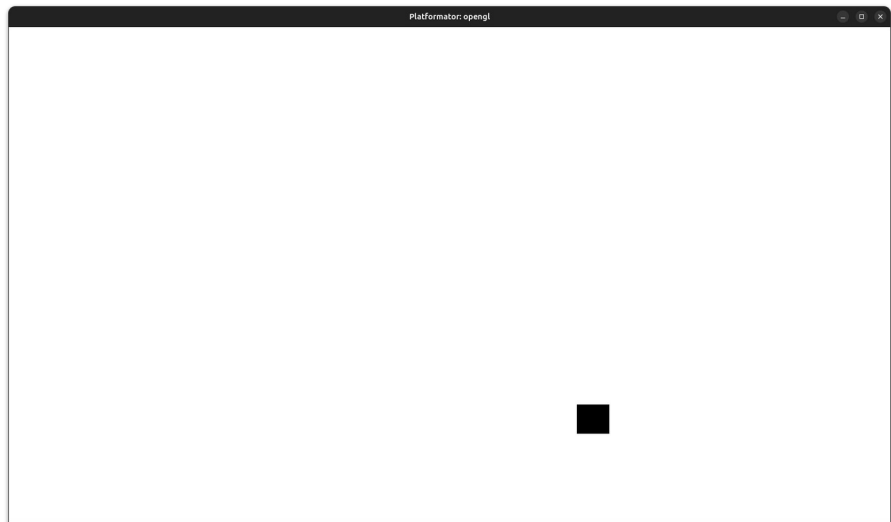
5.3 Scenariji za merenje performansi

Korišćeni su sledeći scenariji za merenje performansi:

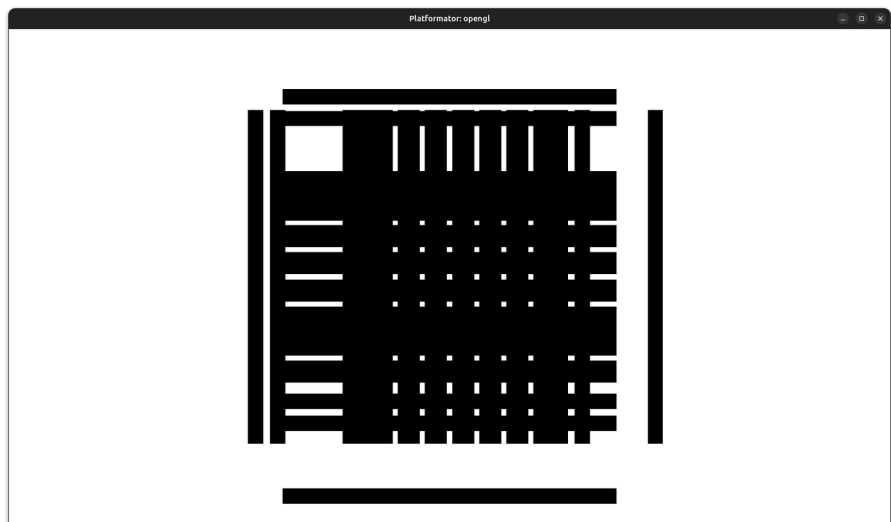
1. Scenario intenzivnih promena u širokoj fazi (visok broj migracija i ažuriranja intervala, dosta pomeranja objekata koji se međusobno ne dodiruju).



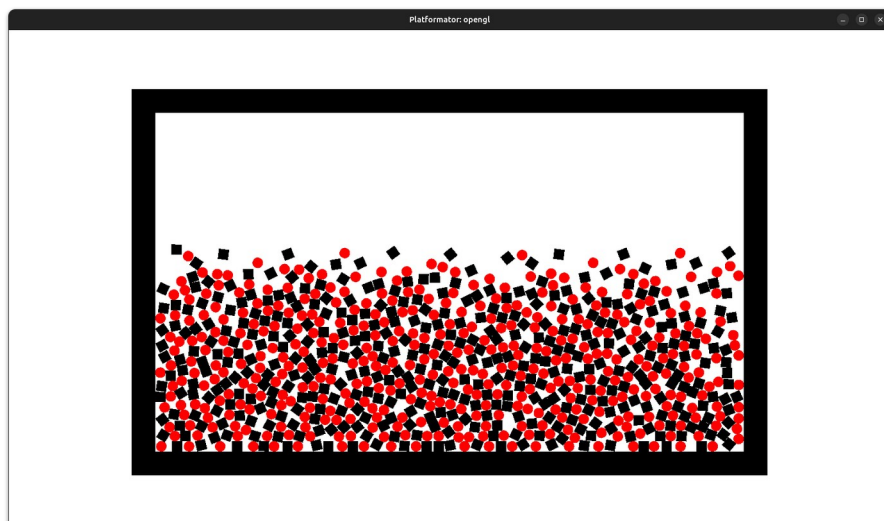
2. Scenario teleportovanja objekata (veliki broj objekata se teleportuje, bez dodirivanja, izaziva velike i nagle promene u širokoj fazi).



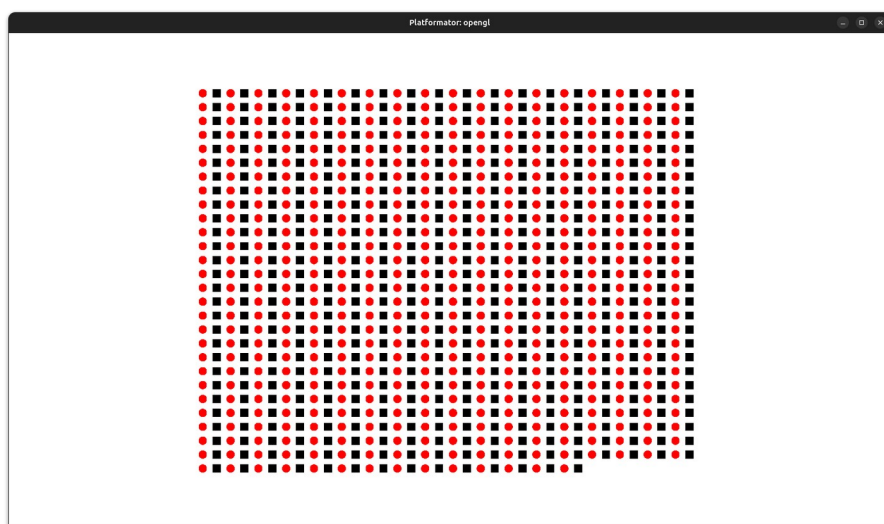
3. Scenario zasićenja uske faze (veliki broj kandidata za usku fazu).



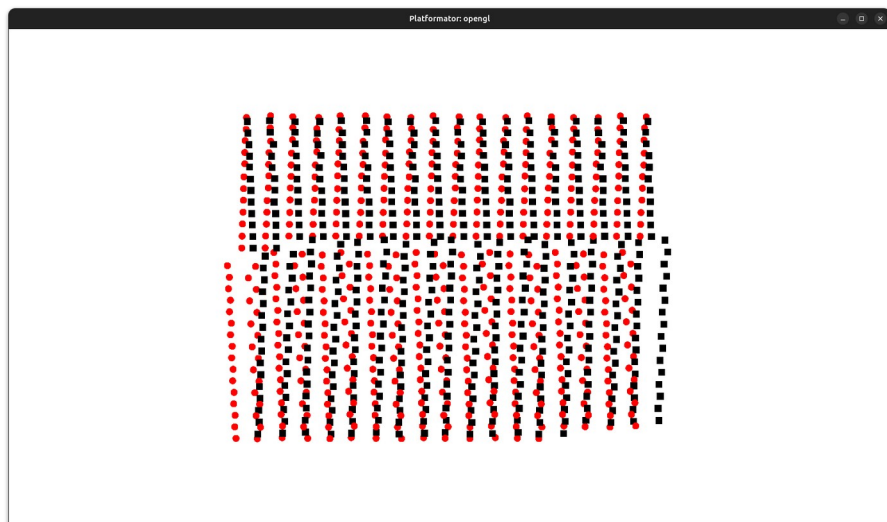
4. Scenario mnogobrojnih dinamičnih tela u posudi (veliki broj rešavanja kolizija).



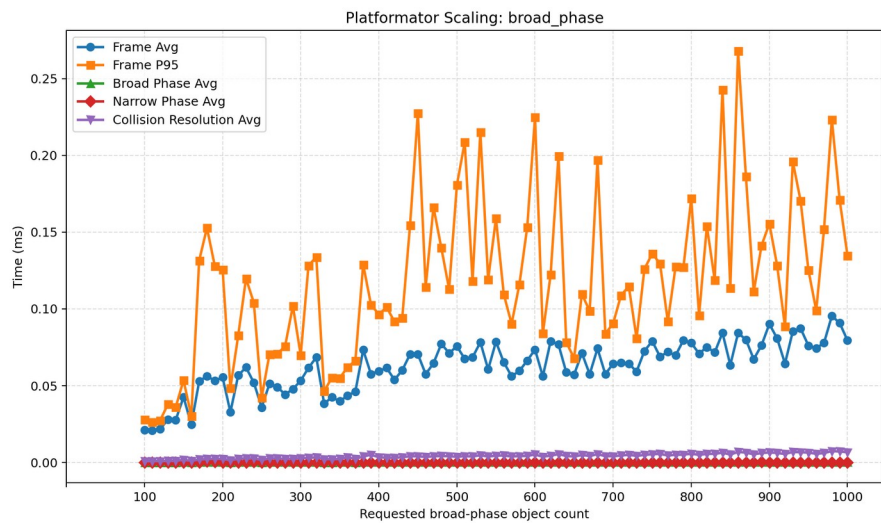
5. Scenario velikog broja dodavanja i brisanja statičkih elemenata.



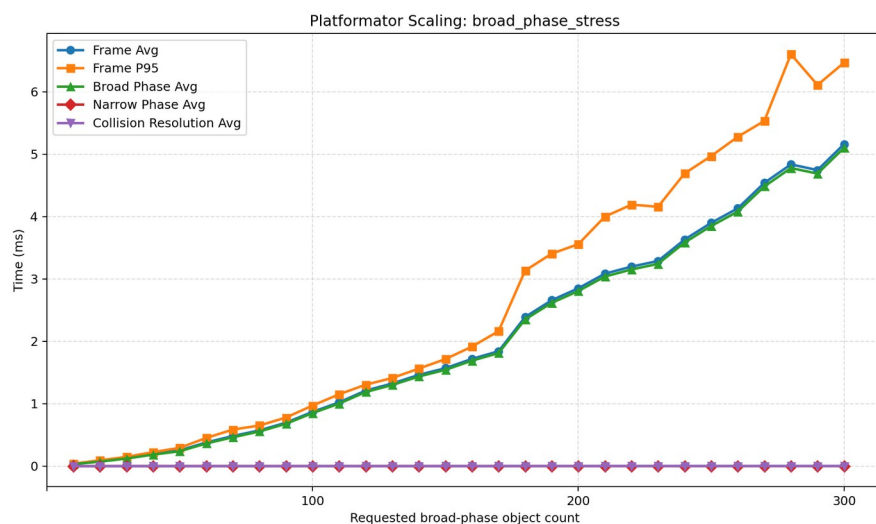
6. Scenario velikog broja dodavanja i brisanja elemenata koji se kreću.



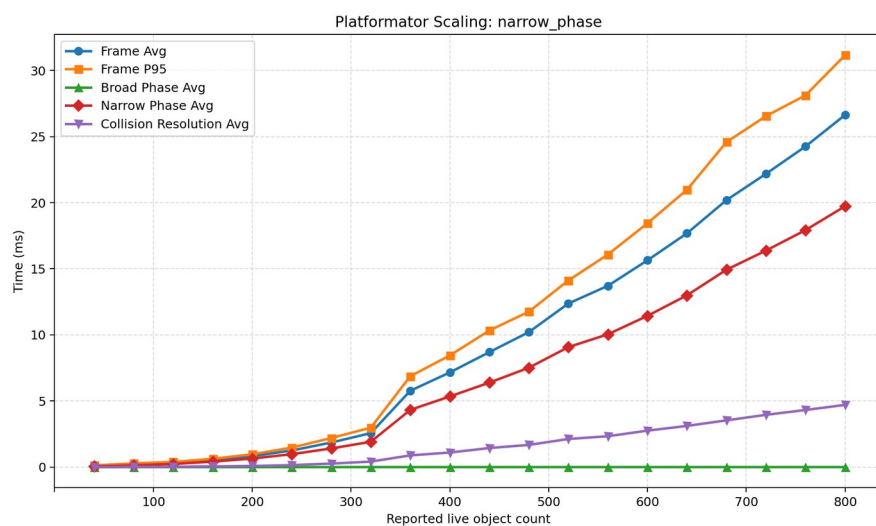
5.4 Rezultati



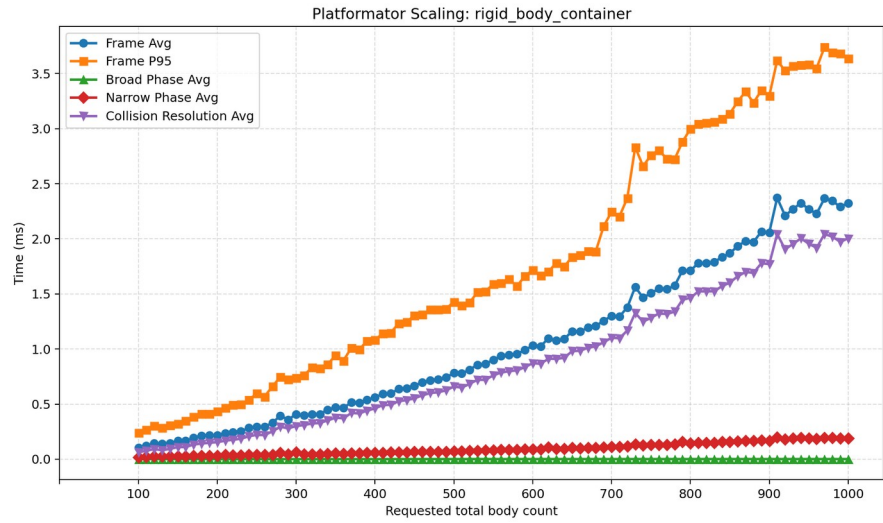
Slika 5.1. Rezultati merenja prvog scenarija (široka faza)



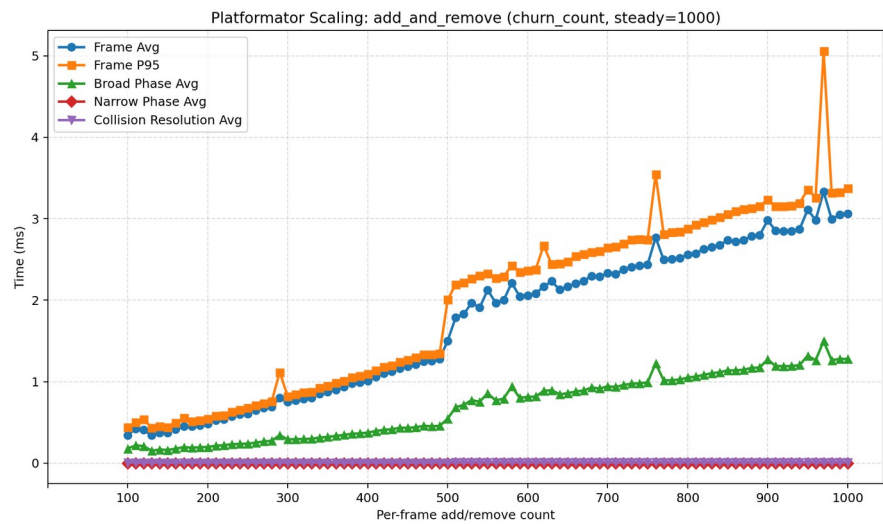
Slika 5.2 Rezultati merenja drugog scenarija (široka faza)



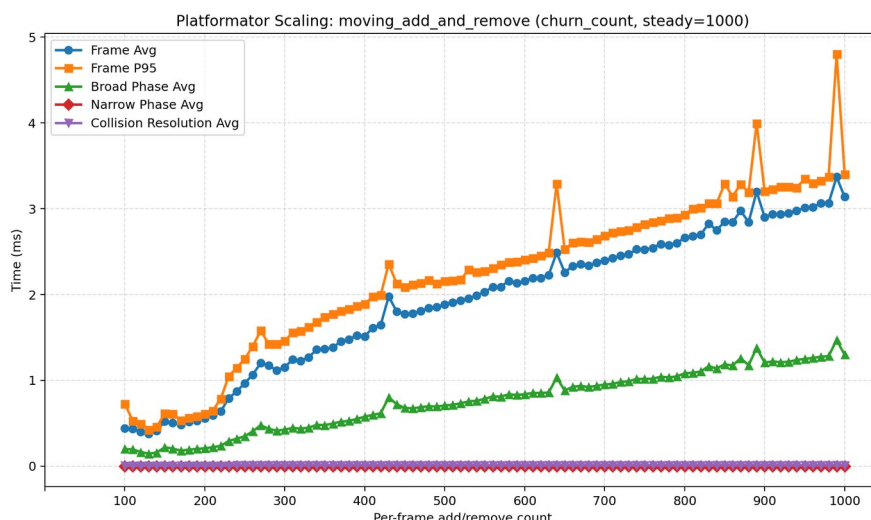
Slika 5.3. Rezultati merenja trećeg scenarija (uska faza)



Slika 5.4. Rezultati merenje četvrtog scenarija (dinamična tela)



Slika 5.5 Rezultati merenja petog scenarija (dodavanje i brisanje)



Slika 5.6 Rezultati merenja šestog scenarija (dodavanje i brisanje)

5.5 Diskusija rezultata

- Široka faza – za široku fazu, očekivano ponašanje je $O(u+e)$ gde je „u“ broj pomeranja, a „e“ broj promena u preklapanju. Na prvom grafikonu liniju za široku fazu skoro pa ne možemo da vidimo, to je usled činjenice da se objekti koji se nalaze u istoj ćeliji kreću u istom pravcu, istom brzinom – stoga je broj promena u preklapanju skoro ne postojeći, dok je broj pomeranja u jednoj ćeliji minimalan. Performanse celog sistema na ovom grafu deluju da se skaliraju linearno.

Treba naglasiti da skok u vremenu izvršavanja koji se prikazuje na prvom grafikonu je usled akcija koje ne spadaju pod široku fazu (kretanje samo po sebi, sinhronizacija projekcija sa kretanjem se vrši odvojeno, u širokoj fazi).

Na drugom grafikonu možemo primetiti da je skaliranje, očevidno, kvadratno. Ovo je usled činjenice da se svi objekti nalaze u istoj ćeliji, zatim se naglo teleportuju u drugu ćeliju, ovo dovodi do kompletnog resortiranja objekata i upoređivanje svakog objekta sa svakim. Treba naglasiti da je ovo najgori slučaj široke faze, naročito za naš algoritam koji se oslanja na prostornu koherenciju.

- Uska faza – za usku fazu, očekivano ponašanje je $O(n^2)$ usled činjenice da je potrebno proveriti svaki objekat sa svakim. Na osnovu grafikona deluje kao da je skaliranje kvadratno, što je ono što smo očekivali.

- Dinamična tela – za test sa velikim brojem odmarajućih dinamičnih tela nismo imali neka definisana očekivanja. Gledajući graf možemo da primetimo da je glavno usko grlo u ovom scenariju algoritam za rešavanje kolizija (sekvencijalni impulsi). Skaliranje deluje linearno $O(n)$.

Nagli skok koji se može primetiti na oko 700 objekata je usled hardverskih ograničenja laptopa koji je korišćen za testiranje.

- Statičko dodavanje i brisanje – za test sa velikim dodavanjem i brisanjem za sisteme gde se puno elemenata kreće, očekivano ponašanje je $O(k(m \log n))$ gde je „k“ broj dimenzija, „m“ broj migracija (pisanja i brisanja), a „n“ broj elemenata – ili ti, očekivali smo linearistično ponašanje. U ovom eksperimentu mi smo držali konstantan broj objekata (1000) dok smo menjali samo broj pisanja i brisanja, time smo očekivali linearne promene (izolovali smo „m“ parametar).

Gledajući graf, deluje kao da se performanse skaliraju linearno, ili možda čak i logaritamski. Takođe možemo da primetimo tokom poređenja grafa gde je bilo pomeranja, i gde nije bilo pomeranja, da su grafovi skoro pa identični, što je ponovo, bolje nego očekivano čak.

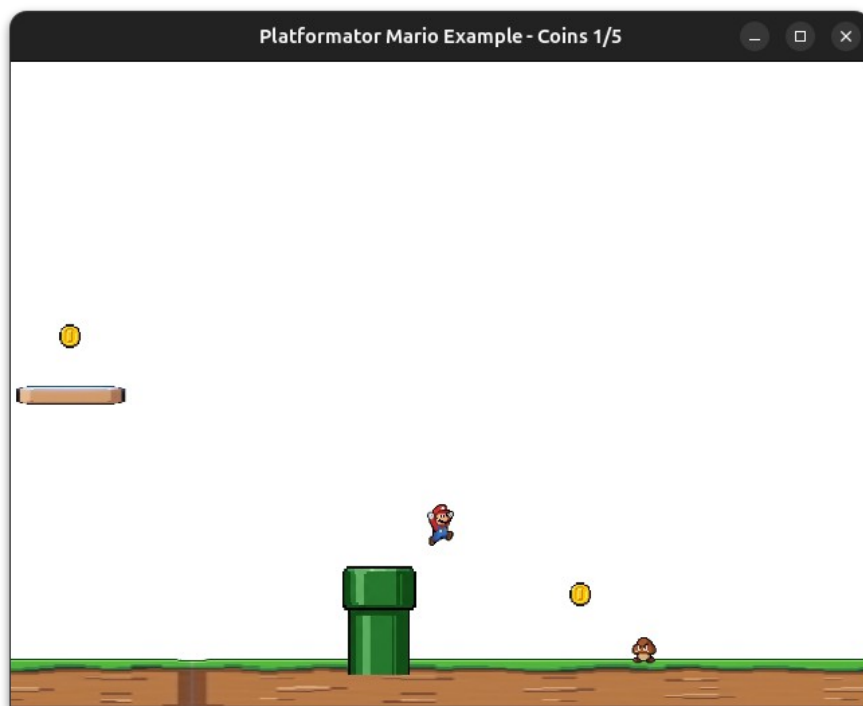
Uočena ograničenja:

- potrebne dodatne optimizacije u užoj fazi (poput uspavljivanja dinamičkih objekata),
- potrebne dodatne optimizacije tokom rešavanja kolizija,

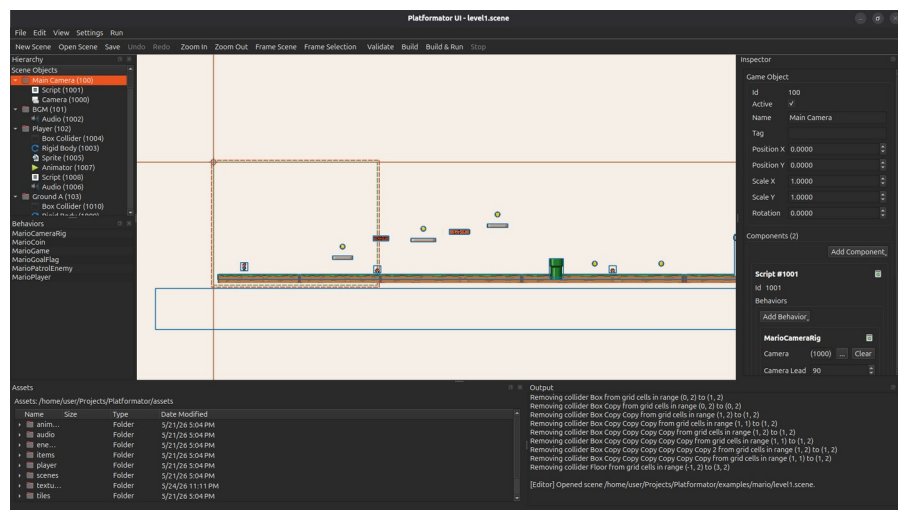
6. DEMONSTRACIJA IMPLEMENTIRANOG SISTEMA

Za demonstraciju je razvijena Mario-like 2D scena koja koristi sve ključne podsisteme motora za igre:

- sprite i animacione komponente,
- audio događaje,
- okidačke i fizičke kolizije,
- skriptno upravljanje ponašanjem igrača i neprijatelja,
- serijalizaciju i učitavanje scene.



Slika 6.1 Prikazuje demonstraciju Mario-like primera.



Slika 6.2 Prikazuje izgled Platformator aplikacije, kao i same Mario scene u njoj.

Takođe je implementiran i program za upravljanjem scena, pisan u python programskom jeziku (prikazan na slici 6.2)

Demonstracija pokazuje da sistem može da podrži realan ciklus igrivosti sa detekcijom i rešavanjem kolizija u realnom vremenu. Istovremeno, scena je upotrebljena kao integracioni test jer aktivira većinu funkcionalnosti koje su pojedinačno testirane regresionim testovima.

7. ZAKLJUČAK

U ovom radu predstavljen je razvoj 2D komponentnog motora za igre Platformator sa fokusom na optimizovanu detekciju kolizija u širokoj fazi. Arhitektura sistema zasnovana je na jasnom razdvajanju odgovornosti kroz komponente i menadžere, uz integraciju biblioteka SDL3, Eigen i nlohmann/json.

Glavni doprinos rada je praktična implementacija podsistema široke faze zasnovanog na segmentisanim intervalnim listama i podeli prostora na ćelije, inspirisana radom Tracy, Buss i Woods. Takav pristup je omogućio efikasnije upravljanje kolizionim kandidatima u scenama sa velikim brojem dinamičkih promena.

Eksperimentalna evaluacija kroz benchmark testove potvrđuje da predložena arhitektura predstavlja stabilnu i performansno opravdanu osnovu za dalji razvoj motora.

Pravci daljeg rada:

1. Menjanje arhitekture, bolja paralelizacija – možda čak i korišćenje flecs-a.
2. Bolji algoritam za uspavljivanje dinamičnih objekata.
3. Korišćenje boljeg algoritma od primitivnih sekvencijalnih impulsa.
4. Proširenje editor i tooling sloja za lakše autorstvo scena.
5. Dodavanje WebGL build-a, kao i mogućnost build-ovanja za više različitih operativnih sistema, kao i za različite arhitekture (android).
6. Promena na 3D.
7. Dodavanje shader-a.
8. Dodavanje networking sloja.
9. Dodavanje joint-ova.

8. LITERATURA

- [1] D. Baraff, Dynamic Simulation of Non-Penetrating Rigid Bodies, PhD Thesis, Cornell University, 1992.
- [2] J. D. Cohen, M. C. Lin, D. Manocha, M. K. Ponamgi, i-COLLIDE: Interactive and Exact Collision Detection for Large-Scale Environments, 1995.
- [3] D. J. Tracy, S. R. Buss, B. M. Woods, Efficient Large-Scale Sweep and Prune Methods with AABB Insertion and Removal, IEEE VR, 2009.
- [4] G. van den Bergen, Collision Detection in Interactive 3D Environments, Morgan Kaufmann, 2003.
- [5] Unity Technologies, Unity Manual and Scripting API, dostupno online.
- [6] Godot Engine Documentation, dostupno online.
- [7] SDL3 Official Documentation, dostupno online.
- [8] Eigen Documentation, dostupno online.
- [9] nlohmann/json Documentation, dostupno online.
- [10] oneTBB Documentation, dostupno online.

9. BIOGRAFIJA

Dragan Arsić je rođen u Požarevcu, gde je završio osnovnu i srednju školu. Studije na Fakultetu tehničkih nauka Univerziteta u Novom Sadu, studijski program Softversko inženjerstvo i informacione tehnologije, upisao je školske 2018 godine.

KLJUČNA DOKUMENTACIJSKA INFORMACIJA

Redni broj, RBR:	
Identifikacioni broj, IBR:	
Tip dokumentacije, TD:	monografska publikacija
Tip zapisa, TZ:	tekstualni štampani dokument
Vrsta rada, VR:	Diplomski rad
Autor, AU:	Dragan Arsić
Mentor, MN:	dr [ime mentora], [zvanje mentora – docent, vanredni profesor ili redovni profesor]
Naslov rada, NR:	Implementacija 2D komponentnog motora za igre sa optimizovanom širokom fazom detekcije kolizija
Jezik publikacije, JP:	srpski
Jezik izvoda, Jl:	srpski
Zemlja publikovanja, ZP:	Srbija
Uže geografsko područje, UGP:	Vojvodina
Godina, GO:	2026
Izdavač, IZ:	autorski reprint
Mesto i adresa, MA:	Novi Sad, Fakultet tehničkih nauka, Trg Dositeja Obradovića 6
Fizički opis rada, FO:	br. poglavlja / stranica / citata / tabela / slika / grafikona / priloga
Naučna oblast, NO:	Softversko inženjerstvo i informacione tehnologije
Naučna disciplina, ND:	Mašinsko učenje/Istraživanje podataka
Predmetna odrednica / ključne reči, PO:	3-5 ključnih reči koje biste koristili u pretraživaču da nađete rad sa ovom temom (npr., mašinsko učenje, konvolucione neuronske mreže)
UDK	
Čuva se, ČU:	Biblioteka Fakulteta tehničkih nauka, Trg Dositeja Obradovića 6, Novi Sad
Važna napomena, VN:	
Izvod, IZ:	apstrakt – jedan pasus koji dobro opisuje suštinu rada – problem, motivaciju, naznaku rešenja i rezultat.
Datum prihvatanja teme, DP:	
Datum odbrane, DO:	
Članovi komisije, KO:	
predsednik	dr Ime Prezime, zvanje
član	dr Ime Prezime, zvanje
mentor	dr Ime Prezime, zvanje
Potpis mentora	

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	monographic publication
Type of record, TR :	textual material
Contents code, CC :	[bachelor or master] thesis
Author, AU :	Ime i prezime kandidata
Mentor, MN :	Ime i prezime mentora, [zvanje - assistant professor, associate professor ili full professor], PhD
Title, TI :	Naslov rada na engleskom
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian / English
Country of publication, CP :	Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2021
Publisher, PB :	author's reprint
Publication place, PP :	Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6
Physical description, PD :	br. poglavlja / stranica / citata / tabela / slika / grafikona / priloga
Scientific field, SF :	Software Engineering and Information Technologies
Scientific discipline, SD :	Machine Learning/Data Mining
Subject / Keywords, S/KW :	Ključne reči na engleskom
UDC	
Holding data, HD :	Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad
Note, N :	
Abstract, AB :	Prevod apstrakta na engleski
Accepted by sci. Board on, ASB :	
Defended on, DE :	
Defense board, DB :	
president	Ime i prezime, zvanje na eng., PhD
member	Ime i prezime, zvanje na eng., PhD
mentor	Ime i prezime, zvanje na eng., PhD
	Mentor's signature